NASA Space Apps 2026 - 14 Challenges Breakdown & Selection Guide

This guide is curated for hackathon participants, team leads, mentors, and strategists entering the NASA Space Apps Challenge 2026. It breaks down all 14 official challenges into clear, actionable blueprints—translating NASA's problem statements into concrete product deliverables, tech stacks, and team profiles.

Earth Observation & Climate Data

1. Be An Earth System Trend Detective!

- Challenge Title & Category: Be An Earth System Trend Detective! | Earth Science & Software
- Difficulty Level: Advanced
- Core Problem: Earth's interconnected climate variables often show conflicting regional behaviors—such as simultaneous extreme drought in one zone and increased rainfall in another—driven by the same overarching global process. Scientists and regional planners lack streamlined tools to quickly visualize these temporal trends and verify whether observed shifts are statistically significant.
- What to Build: An interactive spatial-temporal analytics dashboard and geospatial analysis engine that ingests multi-decade NASA Earth observations, executes statistical trend detection (e.g., Mann-Kendall tests and Sen's slope estimation), and displays regional divergence maps with interactive time-series charts.
- Recommended Tech Stack: Python (FastAPI, GeoPandas, xarray, rasterio, SciPy/statsmodels), Google Earth Engine API, React, Mapbox GL / Deck.gl, Recharts / Plotly.js.
- Who Should Choose This: Best for data scientists, geospatial researchers, and full-stack developers skilled in time-series statistics, raster data processing, and performant web mapping.

2. Dancing with the SARs

- Challenge Title & Category: Dancing with the SARs | Earth Science, Remote Sensing & Radar
- Difficulty Level: Intermediate / Advanced
- Core Problem: Earth's surface undergoes constant subtle deformation from earthquakes, volcanic activity, groundwater depletion, and deforestation, but radar remote sensing data is notoriously unintuitive for non-specialists to analyze. With the NASA-ISRO Synthetic Aperture Radar (NISAR) mission, massive volumes of dual-frequency radar data require automated processing to translate complex interferograms into actionable surface displacement insights.
- What to Build: An interactive radar remote sensing visualization portal and displacement analytics tool that processes NISAR and Sentinel-1 InSAR data to help users track millimeter-scale ground shifts, wetland changes, and post-disaster land deformation.
- Recommended Tech Stack: Python (GDAL, rasterio, MintPy, ISCE3), Google Earth Engine API, Next.js, FastAPI, MapLibre GL / Deck.gl, Cloud-Optimized GeoTIFFs (COG), STAC APIs.
- Who Should Choose This: Best for GIS engineers, remote sensing specialists, radar analysts, and frontend developers passionate about planetary-scale Earth monitoring.

3. Field Shift: Adapting Farms with NASA Data

- Challenge Title & Category: Field Shift: Adapting Farms with NASA Data | Agriculture, Earth Science & Software
- Difficulty Level: Intermediate / Advanced
- Core Problem: Agricultural producers worldwide face severe rainfall volatility, shifting seasonal temperatures, and declining soil fertility, making multi-season crop rotation planning risky and uncertain. Farmers lack localized, accessible decision-support tools that integrate satellite soil moisture, evapotranspiration, and climate forecasts with farm-specific agronomic constraints.
- What to Build: A localized crop rotation advisor and mobile/web dashboard that combines NASA Earth observations (SMAP soil moisture, MODIS/Landsat vegetation indices, GPM precipitation, OpenET) with local soil health metrics to generate resilient crop rotation calendars and risk-reduction strategies.

- Recommended Tech Stack: Python (FastAPI, Pandas, XGBoost / LightGBM for yield & risk forecasting), NASA POWER API, NASA Earthdata / AppEEARS APIs, React / Next.js (with Progressive Web App support for offline field use), Leaflet / MapLibre GL.
- Who Should Choose This: Best for teams blending agronomy and climate science with tabular machine learning experience, full-stack web development, and user-centric product design.

4. Harmonization of MODIS and VIIRS Hot Spots

- Challenge Title & Category: Harmonization of MODIS and VIIRS Hot Spots | Earth Science, Thermal Anomalies & Software
- Difficulty Level: Intermediate / Advanced
- Core Problem: Satellite instruments have tracked planetary active fire hotspots for over two decades, but the record is split between legacy MODIS sensors (1 km resolution) and newer VIIRS sensors (375 m resolution) with different detection sensitivities and overpass times. Fire managers and researchers lack a unified, cross-calibrated burning calendar, making it difficult to discern true wildfire regime shifts from sensor artifacts.
- What to Build: A cross-sensor fire data harmonization engine and interactive burning activity calendar that ingests NASA FIRMS data, applies spatial clustering and statistical calibration to reconcile sensor discrepancies, and delivers standardized wildfire trend analyses.
- Recommended Tech Stack: Python (DuckDB / Spatial SQL, GeoPandas, NumPy, scikit-learn), NASA FIRMS API / Global Fire Atlas, FastAPI, MapLibre GL / Deck.gl, Apache Arrow / Parquet, React.
- Who Should Choose This: Best for big-data engineers, database architects, geospatial analysts, and backend developers experienced with large-scale spatial queries and sensor calibration.

5. The Earth Information Jukebox

- Challenge Title & Category: The Earth Information Jukebox | Arts & Multimedia, Earth Science & Software
- Difficulty Level: Beginner / Intermediate
- Core Problem: Visual representations of climate datasets—such as global temperature anomalies and greenhouse gas plumes—are often inaccessible to visually impaired individuals and can feel emotionally

detached for the broader public. Sonification translates complex environmental time-series into auditory experiences, creating an intuitive, multi-sensory connection to Earth's vital signs.

- What to Build: An interactive web application ("Earth Jukebox") that synchronizes NASA Earth Information Center (EIC) visual frames with real-time, data-driven audio synthesis (e.g., mapping atmospheric CO2 levels to harmonic tension, ocean temperatures to pitch frequencies, and ice sheet loss to rhythmic decay).
- Recommended Tech Stack: Web Audio API, Tone.js, React / Svelte, D3.js / HTML5 Canvas, NASA Global Climate Change & EIC data feeds, ARIA accessibility standards, Tailwind CSS.
- Who Should Choose This: Best for creative technologists, audio designers, music programmers, accessibility advocates, and frontend developers interested in experiential data art.

Space Exploration & Planetary Missions

6. CLPS Lunar Mission Browser

- Challenge Title & Category: CLPS Lunar Mission Browser | Human Exploration, Moon, Software & Space Exploration
- Difficulty Level: Intermediate / Advanced
- Core Problem: Commercial Lunar Payload Services (CLPS) missions heading to the lunar south pole must navigate severe low-angle solar illumination and intermittent direct-to-Earth radio line-of-sight conditions. Existing ephemeris and trajectory tools are complex and academic, preventing mission planners, educators, and the public from rapidly assessing landing site viability.
- What to Build: An intuitive 3D lunar mission exploration portal and landing site browser that visualizes Sun and Earth elevations relative to the local horizon across candidate landing dates, calculating solar power availability and direct-to-Earth communication windows.
- Recommended Tech Stack: CesiumJS / Three.js, NASA SPICE toolkit (WebGeocalc REST API / SpiceyPy), NASA Planetary Data System (PDS) LOLA lunar digital elevation models, React / Next.js, Tailwind CSS, Plotly.js.

- Who Should Choose This: Best for aerospace enthusiasts, 3D graphics developers, orbital mechanics students, and frontend engineers passionate about lunar exploration.

7. Create Health Monitoring Software for Astronauts on Space Missions

- Challenge Title & Category: Create Health Monitoring Software for Astronauts on Space Missions | Human Exploration, Software & Space Exploration
- Difficulty Level: Intermediate
- Core Problem: Long-duration spaceflight exposes astronauts to deep space radiation, microgravity-induced bone and muscle deterioration, cardiovascular deconditioning, and cognitive fatigue, with communication latency preventing real-time Earth support. Astronauts must take personal responsibility for monitoring biometric indicators and executing timely countermeasures autonomously.
- What to Build: An autonomous, offline-first digital astronaut health dashboard and clinical decision support system that ingests wearable biometric telemetry, performs anomaly detection on health metrics, and recommends targeted countermeasures (e.g., specialized exercise protocols, nutritional adjustments, and sleep schedule changes).
- Recommended Tech Stack: TypeScript, React / React Native, Python (FastAPI), scikit-learn (physiological anomaly detection and trend forecasting), SQLite / IndexedDB (offline-first edge architecture), Chart.js / Recharts, Tailwind CSS.
- Who Should Choose This: Best for health-tech developers, biomedical engineers, UI/UX designers, and mobile/web engineers passionate about bio-astronautics and human performance monitoring.

8. Identify Earth Locations that Analog the Permanent Moon Base Locations and Mars

- Challenge Title & Category: Identify Earth Locations that Analog the Permanent Moon Base Locations and Mars | Earth Science, Planets & Moons, Software & Space Exploration
- Difficulty Level: Beginner / Intermediate / Advanced
- Core Problem: Preparing rovers, habitats, and crew procedures for lunar polar craters and Martian plains requires extensive testing at terrestrial

analog environments that mirror planetary geology, terrain roughness, and environmental stress. However, existing analog catalogs are fragmented and lack quantitative multi-parameter scoring that correlates planetary conditions with accessible Earth sites.

- What to Build: A multi-criteria geospatial matching engine and interactive analog explorer that cross-references lunar and Martian remote sensing layers (LRO LOLA, MRO CRISM/HiRISE) with Earth observation data (ASTER, Landsat, Sentinel, USGS mineralogy) to compute objective "Analog Similarity Scores" for field testing.
- Recommended Tech Stack: Python (GeoPandas, SciPy, scikit-learn for multi-attribute matching and clustering), Google Earth Engine / Microsoft Planetary Computer, Mapbox GL / Leaflet, React, Tailwind CSS, FastAPI.
- Who Should Choose This: Best for planetary scientists, geology students, GIS analysts, and full-stack web developers keen to compare planetary datasets.

9. Interplanetary Survival Guide: Martian Map

- Challenge Title & Category: Interplanetary Survival Guide: Martian Map | Human Exploration, Mars, Planets & Moons, Software & Space Exploration
- Difficulty Level: Intermediate
- Core Problem: Future Martian explorers traversing the surface will face steep terrain hazards, abrasive dust storms, and extreme cold without access to Earth GPS or immediate mission control guidance. Astronauts require an integrated tactical navigation system that unites topographical slopes, ground trafficability, scientific sample targets, and localized dust storm monitoring into a single interface.
- What to Build: An interactive, multi-layered 3D Martian tactical navigation and route-planning application that renders high-resolution Martian terrain, evaluates slope safety, highlights high-value scientific targets, and optimizes extravehicular activity (EVA) traverse paths with resource consumption budgets (oxygen, battery, life support).
- Recommended Tech Stack: Three.js / CesiumJS / MapLibre GL 3D, NASA Mars Trek WMTS API, USGS Astrogeology APIs, Python (NetworkX for A* route pathfinding), React, Tailwind CSS.
- Who Should Choose This: Best for 3D web developers, map enthusiasts, pathfinding / robotics algorithm developers, and UI/UX designers creating tactical sci-fi interfaces.

10. Planet X and SPHEREx

- Challenge Title & Category: Planet X and SPHEREx | Astrophysics, Planets & Moons & Software
- Difficulty Level: Advanced
- Core Problem: NASA's SPHEREx space observatory maps the entire celestial sky every six months across 102 near-infrared spectral bands, collecting photometric spectra for hundreds of millions of cosmic objects. Uncovering faint moving bodies (such as Kuiper Belt objects or hypothetical outer solar system planets), brown dwarfs, and variable stars requires sifting through immense multi-epoch spectral survey cubes.
- What to Build: A high-performance celestial sky survey visualizer, blink comparator, and candidate transient-detection pipeline that enables professional astrophysicists and citizen scientists to browse SPHEREx all-sky spectral layers, track moving objects between observation epochs, and flag orbital anomaly candidates.
- Recommended Tech Stack: Python (Astropy, photutils, NumPy, CuPy / Numba for accelerated array math), Aladin Lite / WorldWide Telescope Web API, WebGL / HTML5 Canvas, FastAPI, React / Next.js.
- Who Should Choose This: Best for astrophysics students, scientific computing engineers, computer vision developers, and web developers experienced with astronomical data and large-scale raster rendering.

AI, NLP & Knowledge Systems

11. Flame in Freefall: AI-Powered Fire Safety Insights from Microgravity Combustion Data

- Challenge Title & Category: Flame in Freefall: AI-Powered Fire Safety Insights from Microgravity Combustion Data | Space Exploration, Artificial Intelligence & Machine Learning
- Difficulty Level: Intermediate / Advanced
- Core Problem: Decades of microgravity combustion experiments (e.g., BASS, ACME, FLEX) have produced critical insights into how fuels ignite, spread, and extinguish in low gravity. However, these vital safety findings are scattered across legacy PDF reports, research papers, and

experimental video logs, making it difficult for spacecraft architects to rapidly extract flammability limits and suppression protocols.
- What to Build: An AI-powered combustion safety portal and retrieval-augmented generation (RAG) knowledge engine that ingests NASA microgravity combustion literature and experimental datasets, extracts key safety metrics (quench diameters, flame propagation rates, limiting oxygen indices), and provides interactive conversational search with safety verification.
- Recommended Tech Stack: Python, LangChain / LlamaIndex, ChromaDB / Qdrant (vector database), OpenAI / Anthropic / Gemini API, FastAPI, Streamlit or React / Vite frontend, Plotly.js for combustion curve visualization.
- Who Should Choose This: Best for AI/NLP engineers, LLM application builders, aerospace/mechanical engineers, and full-stack developers skilled in document extraction, embeddings, and domain-specific knowledge assistants.

Interactive Games & Storytelling

12. Abandoned but not Forgotten: Storytelling about NASA's Discarded Equipment on the Moon and Mars

- Challenge Title & Category: Abandoned but not Forgotten: Storytelling about NASA's Discarded Equipment on the Moon and Mars | Astrophysics, Planets & Moons, Space Exploration & Storytelling
- Difficulty Level: Beginner / Intermediate
- Core Problem: Over more than six decades of deep space exploration, NASA has left hardware—from Apollo descent stages and lunar roving vehicles to inactive Mars rovers and deep space probes—scattered across our solar system. School-age learners and the general public often view these artifacts simply as space junk rather than monuments to scientific breakthroughs.
- What to Build: An interactive digital storytelling museum, gamified 3D virtual tour, or narrative-driven web experience that introduces students to historical space hardware, their final resting sites, and the groundbreaking science they achieved.

- Recommended Tech Stack: Three.js / React Three Fiber, Next.js, Tailwind CSS, Blender (for lightweight 3D models), Web Audio API / ElevenLabs for voiceover narration, NASA 3D Models / Historical Archives.
- Who Should Choose This: Best for creative writers, UI/UX designers, 3D frontend developers, educators, and multimedia artists who want to create an emotionally resonant, visually rich web experience.

13. Build a Junior Astronaut Mission Trainer

- Challenge Title & Category: Build a Junior Astronaut Mission Trainer | Games, Planets & Moons, Software, Space Exploration & Sun
- Difficulty Level: Beginner / Intermediate
- Core Problem: Space education content frequently oversimplifies the engineering compromises that define planetary exploration or presents them through dry mathematical formulas that disengage young students. Few tools make the delicate balance of life support, power generation, radiation shielding, and food production both tangible and fun.
- What to Build: An interactive simulation management game (playable on web browsers or tablets) where young players oversee a lunar or Martian outpost, balancing competing resources (solar/nuclear energy, oxygen, water recycling, crop growth) while responding to solar flare alerts and mechanical breakdowns.
- Recommended Tech Stack: Unity (WebGL / C#), Godot 4 (GDScript / Web export), or web-native game engines (Phaser 3 / Pixi.js + TypeScript), Tailwind CSS, Howler.js for sound effects.
- Who Should Choose This: Best for indie game developers, 2D/3D artists, game designers, and educational technology enthusiasts aiming to build a polished, replayable game loop with clear educational impact.

14. Space Mission Design Game

- Challenge Title & Category: Space Mission Design Game | Games & Simulation
- Difficulty Level: Beginner / Intermediate
- Core Problem: Formulating a real space mission requires balancing difficult architectural trade-offs between scientific objectives, payload mass, launch vehicle delta-v, communications latency, electrical power, and strict cost caps. Students and space enthusiasts rarely get the

opportunity to experiment with these systems-engineering trade-offs in an approachable, hands-on environment.

- What to Build: An interactive space mission architecture and tycoon simulator where players configure complete missions from concept to launch—selecting destinations (e.g., Europa, Titan, Mars), choosing instruments, balancing mass and budget constraints, and simulating mission execution.
- Recommended Tech Stack: Unity (WebGL), Godot 4, or React / Canvas / Pixi.js, Tailwind CSS, Chart.js for mission telemetry and system resource charts.
- Who Should Choose This: Best for game developers, systems simulation enthusiasts, UI designers, and engineering students interested in mission design, aerospace economics, and gamification.

Strategic Decision Matrix & Comparison Table

Challenge Title	Focus Category	Difficulty Level	Key NASA Data / APIs	Primary Deliverable	Ideal Team Profile
Be An Earth System Trend Detective!	Earth Observation & Climate Data	Advanced	NASA Earthdata, GISTEMP, CERES	Statistical Trend Dashboard	Data Scientists, Climate Researchers, Geospatial Devs
Dancing with the SARs	Earth Observation & Climate Data	Intermediate / Advanced	NISAR, Sentinel-1 InSAR, Alaska Satellite Facility	Radar Surface Change Map	GIS Specialists, Radar Analysts, Frontend Developers
Field Shift: Adapting Farms with NASA Data	Earth Observation & Climate Data	Intermediate / Advanced	NASA POWER, SMAP, OpenET, MODIS	Crop Rotation Advisory App	ML Engineers, Agronomists, Full-Stack / Mobile Devs
Harmonization of MODIS and VIIRS Hot Spots	Earth Observation & Climate Data	Intermediate / Advanced	NASA FIRMS (MODIS & VIIRS active fire)	Unified Fire Activity Calendar	Big Data Engineers, Database Architects, Geospatial Devs

Challenge Title	Focus Category	Difficulty Level	Key NASA Data / APIs	Primary Deliverable	Ideal Team Profile
The Earth Information Jukebox	Earth Observation & Climate Data	Beginner / Intermediate	NASA Earth Information Center (EIC) data	Data Sonification Synthesizer	Creative Coders, Audio Engineers, Accessibility Advocates
CLPS Lunar Mission Browser	Space Exploration & Planetary Missions	Intermediate / Advanced	NASA SPICE (WebGeocalc), LOLA lunar DEMs	3D Mission & Horizon Visualizer	Orbital Mechanics Devs, 3D Web Engineers, Frontend Devs
Create Health Monitoring Software for Astronauts	Space Exploration & Planetary Missions	Intermediate	NASA Life Sciences Data Archive (LSDA)	Astronaut Health Dashboard	Health-Tech Engineers, Biomedical Devs, UI/UX Designers
Identify Earth Locations that Analog Moon/Mars	Space Exploration & Planetary Missions	Beginner / Intermediate / Advanced	LRO LOLA, MRO CRISM, ASTER, Landsat	Terrestrial Analog Matching Engine	Geologists, Planetary Scientists, GIS Analysts
Interplanetary Survival Guide: Martian Map	Space Exploration & Planetary Missions	Intermediate	NASA Mars Trek, HiRISE, HRSC elevation	3D Martian Navigation Guide	3D Web Developers, Pathfinding / Algorithm Devs
Planet X and SPHEREx	Space Exploration & Planetary Missions	Advanced	SPHEREx all-sky survey, IPAC / IRSA	Sky Survey Blink Comparator	Astrophysicists, Computer Vision Devs, Scientific Devs
Flame in Freefall	AI, NLP & Knowledge Systems	Intermediate / Advanced	NASA Physical Sciences Data (BASS, ACME)	RAG Combustion Knowledge Portal	AI / NLP Engineers, LLM App Builders, Aero Engineers
Abandoned but not Forgotten	Interactive Games & Storytelling	Beginner / Intermediate	NASA 3D Models, History Office Archives	Interactive 3D Museum Experience	Storytellers, 3D Web Designers, Creative Writers
Build a Junior Astronaut Mission Trainer	Interactive Games & Storytelling	Beginner / Intermediate	NASA Space Biology, Artemis STEM Guides	Colony Management Simulation Game	Game Developers, Educational Tech Designers, 2D Artists

Challenge Title	Focus Category	Difficulty Level	Key NASA Data / APIs	Primary Deliverable	Ideal Team Profile
Space Mission Design Game	Interactive Games & Storytelling	Beginner / Intermediate	NASA Eyes on the Solar System, PDS	Mission Architecture Simulator	Game Programmers, Systems Engineers, UI/UX Designers

---

10-Minute Team Selection Checklist

Use this 3-step decision flow during your initial team kickoff to converge on the right challenge in under 10 minutes:

Step 1: Skillset & Passion Audit (Minutes 0–3)

- Map Team Strengths:
  - If your team has strong Tabular ML / Python Data Science / GIS skills → Choose Be An Earth System Trend Detective!, Dancing with the SARs, Field Shift, or Harmonization of MODIS and VIIRS Hot Spots.
  - If your team has strong LLM / RAG / NLP skills → Choose Flame in Freefall.
  - If your team excels at 3D Graphics (Three.js / CesiumJS / WebGL) skills → Choose CLPS Lunar Mission Browser, Interplanetary Survival Guide: Martian Map, or Abandoned but not Forgotten.
  - If your team has Game Engine (Unity / Godot / Phaser) talent → Choose Build a Junior Astronaut Mission Trainer or Space Mission Design Game.
  - If your team prioritizes Multimedia, Sound & Creative Storytelling → Choose The Earth Information Jukebox or Abandoned but not Forgotten.
- Identify Non-Negotiable Passions: Eliminate challenges that no team member is personally excited to work on for 48 hours.

Step 2: Data Complexity & Tooling Evaluation (Minutes 3–6)

- Assess API Accessibility: Can you fetch sample data within 30 minutes?
  - Lightweight APIs: NASA POWER, FIRMS, EIC, and Mars Trek are easy to query via REST.

    - Heavy Datasets: NISAR/SAR radar interferograms and SPHEREx spectral cubes require substantial storage, preprocessing pipelines, and domain knowledge.
- Determine Technical Architecture: Decide whether your deliverable can be a client-side web application (faster to deploy and host during judging) or requires a dedicated cloud backend / GPU inference pipeline.

Step 3: Hackathon Viability & Winning Differentiation (Minutes 6–10)

- Define Your "Magic Moment": Every winning Space Apps project has one memorable hook that captivates judges within the first 30 seconds of a demo video:
    - A visual hook: A responsive 3D globe, animated trajectory, or tactile game interface.
    - A real-world hook: Direct utility for farmers, early responders, or mission planners.
    - An accessibility hook: Engaging underserved communities through data sonification or youth education.
- Vote & Lock-In: Conduct a quick blind vote (each member gets 2 votes). The top-ranked challenge is locked in—begin scoping your minimum viable product (MVP) immediately!